lua-xlsx - English documentation

Julien Freyermuth

Contents

1 lua-xlsx - English documentation

Reference for the `xlsx` and `dataframe` modules. Babet with Lua 5.5 is the primary target, while the pure-Lua core remains compatible with standard Lua 5.3+.

1.1 Contents

1.2 Architecture and Babet integration

`xlsx.lua` and `dataframe.lua` require no external Lua module. When the global `babet` table is available, `xlsx.lua` automatically uses:

Babet function	lua-xlsx use
`babet.writeFileAtomic`	atomic and durable XLSX publication
`babet.archive.test`	full ZIP validation before parsing
`babet.fileSize`	file-size check before loading into memory
`babet.crc32`	native CRC-32 calculation

If one function is absent, the pure-Lua path is retained for that operation.

To disable Babet for one call:

```
assert(wb:save("report.xlsx", { use_babet = false }))

local read, err = xlsx.open("report.xlsx", {
  use_babet = false,
})
assert(read, err)
```

This is mostly useful for testing the fallback. Production code running in Babet should normally keep the default `true`.

1.3 Conventions

- `sheet:write(row, col, value)` and `sheet:read(row, col)` use **0-based** indices. Cell `A1` is `(0, 0)`.
- `sheet:rows()` yields **1-based** Lua arrays. `row[1]` is column A and `row.n` is the width.
- An empty cell is `nil`. Boolean `false` never collapses to `nil`.
- Dates are returned as ISO 8601 strings.
- XLSX bounds are enforced: rows `0..1048575`, columns `0..16383`.
- Written strings must be valid UTF-8 and contain only XML 1.0 characters.
- DataFrame transformations return newly copied row records.

1.4 Writing XLSX files

1.4.1 `xlsx.new([opts]) -> workbook`

Option:

Option	Default	Values
`date_system`	`"1900"`	`"1900"` or `"1904"`

```
local wb1900 = xlsx.new()
local wb1904 = xlsx.new({ date_system = "1904" })
```

Unknown options are rejected.

1.4.2 `workbook:add_sheet([name]) -> sheet`

The default name is `SheetN`.

Rules:

- 1 to 31 **Unicode characters**, not bytes;
- valid UTF-8 and XML 1.0;
- none of `:`, `\`, `/`, `?`, `*`, `[` or `]`;
- no leading or trailing apostrophe;
- no exact duplicate or ASCII case-only duplicate.

```lua
local wb = xlsx.new()
local data = wb:add_sheet("Data")
local unicode = wb:add_sheet(string.rep("é", 20))
```

1.4.3 `sheet:write(row, col, value) -> sheet`

Accepted values: string, finite number, boolean, date value or `nil`.

```lua
local sh = xlsx.new():add_sheet("Example")
sh:write(0, 0, "Text")
sh:write(0, 1, 42)
sh:write(0, 2, 3.14)
sh:write(0, 3, false)
sh:write(0, 4, xlsx.date(2026, 7, 18))
```

NaN, infinities, unsupported types and out-of-range indices raise a Lua error.

1.4.4 `sheet:append_row(values)`

```lua
sh:append_row({ "Alice", 30, true })
sh:append_row({ "Bob", 25, false })
```

1.4.5 `sheet:write_rows(matrix)`

```lua
sh:write_rows({
  { "Name", "Score" },
  { "Alice", 18 },
  { "Bob", 15 },
})
```

1.4.6 `workbook:save(path [, opts])`

Option	Default	Effect
`overwrite`	`true`	replace an existing file
`durable`	`true`	request durable synchronization in Babet
`permissions`	`0644`	final Babet file mode
`use_babet`	`true`	use `writeFileAtomic` when available

Simple write:

```lua
local ok, err = wb:save("report.xlsx")
assert(ok, err)
```

Refuse overwrite:

```lua
local ok, err = wb:save("report.xlsx", {
  overwrite = false,
})
```

Rebuildable cache:

```
assert(wb:save("cache.xlsx", {
  durable = false,
}))
```

Private permissions:

```
assert(wb:save("private.xlsx", {
  permissions = tonumber("600", 8),
}))
```

Combined example:

```
local ok, err = wb:save("export/report.xlsx", {
  overwrite = true,
  durable = true,
  permissions = tonumber("640", 8),
  use_babet = true,
})
assert(ok, err)
```

With Babet, this calls `babet.writeFileAtomic`. Standard Lua writes a temporary file in the same directory, checks `write`, `flush` and `close`, then renames it. The fallback is atomic on ordinary POSIX filesystems but does not duplicate all of Babet's descriptor confinement and `fsync` guarantees.

1.4.7 xlsx.write_rows(path, matrix [, opts])

Options are `sheet`, `date_system`, `overwrite`, `durable`, `permissions` and `use_babet`.

```
assert(xlsx.write_rows("result.xlsx", {
  { "x", "y" },
  { 1, 2 },
  { 3, 4 },
}, {
  sheet = "Results",
  date_system = "1900",
  overwrite = true,
}))
```

1.5 1900 and 1904 dates

```
sh:write(0, 0, xlsx.date(2026, 7, 18))
sh:write(0, 1, xlsx.datetime(2026, 7, 18, 13, 45, 30))
```

The constructors validate real calendar dates, leap years, years `1..9999`, hours `0..23`, minutes and seconds `0..59`, and integer argument types.

Helpers:

```
local serial1900 = xlsx.date_to_serial(2026, 7, 18)
local serial1904 = xlsx.date_to_serial(2026, 7, 18, 0, 0, 0, true)

print(xlsx.serial_to_iso(serial1900, false))
print(xlsx.serial_to_iso(serial1904, false, true))
```

The writer emits the correct serial for the workbook's `date_system`. The reader automatically detects `<workbookPr date1904="1"/>`.

1.6 Reading XLSX files

1.6.1 xlsx.open(path [, opts])

Opening performs:

1. file-size inspection;
2. `babet.archive.test()` validation when available;
3. bounded in-memory loading;
4. central and local ZIP header validation;
5. CRC-checked extraction of the XML parts that are used;
6. workbook, shared-string and style parsing.

```lua
local wb, err = xlsx.open("report.xlsx")
assert(wb, err)
```

Disable only Babet's archive preflight:

```lua
local wb, err = xlsx.open("report.xlsx", {
  validate_archive = false,
})
```

The Lua ZIP checks still apply.

1.6.2 `workbook:date_system()`

Returns `"1900"` or `"1904"`.

1.6.3 `workbook:sheet_names()`

```lua
for _, name in ipairs(wb:sheet_names()) do
  print(name)
end
```

1.6.4 `workbook:sheet(which)`

`which` is a name or a 1-based integer index.

```lua
local first = wb:sheet(1)
local data = wb:sheet("Data")
```

Worksheets are extracted lazily and cached. A worksheet-specific corruption is therefore returned by `sheet()` as `nil, err`.

1.6.5 `sheet:read(row, col)`

Returns a string, number, boolean, ISO date string or `nil`. OOXML error cells are currently returned as strings.

1.6.6 `sheet:dims()`

Returns maximum 0-based row and column indices, or `-1, -1` for an empty sheet.

1.6.7 `sheet:rows()`

```lua
for row in sheet:rows() do
  for col = 1, row.n do
    print(row[col])
  end
end
```

Intermediate empty spreadsheet rows are yielded to preserve row numbering.

1.7 ZIP validation and limits

Option	Default	Maximum
max_file_size	256 MiB	no extra internal ceiling
max_entries	10,000	100,000
max_entry_size	64 MiB	8 GiB
max_total_size	512 MiB	64 GiB
max_path_length	4,096 bytes	1 MiB
max_total_name_bytes	16 MiB	64 MiB
max_compression_ratio	200	1,000,000,000
use_babet	true	boolean
validate_archive	true	boolean

One example per limit:

```
xlsx.open("a.xlsx", { max_file_size = 32 * 1024 * 1024 })
xlsx.open("a.xlsx", { max_entries = 2000 })
xlsx.open("a.xlsx", { max_entry_size = 16 * 1024 * 1024 })
xlsx.open("a.xlsx", { max_total_size = 128 * 1024 * 1024 })
xlsx.open("a.xlsx", { max_path_length = 1024 })
xlsx.open("a.xlsx", { max_total_name_bytes = 1024 * 1024 })
xlsx.open("a.xlsx", { max_compression_ratio = 100 })
```

Combined untrusted-file profile:

```
local wb, err = xlsx.open("import.xlsx", {
  max_file_size = 64 * 1024 * 1024,
  max_entries = 5000,
  max_entry_size = 16 * 1024 * 1024,
  max_total_size = 128 * 1024 * 1024,
  max_path_length = 2048,
  max_total_name_bytes = 2 * 1024 * 1024,
  max_compression_ratio = 100,
  use_babet = true,
  validate_archive = true,
})
assert(wb, err)
```

The Lua reader rejects multi-disk ZIPs, ZIP64, encryption, unsupported methods, unsafe or duplicate names, inconsistent local headers or data descriptors, overlapping entries, size mismatches, CRC failures and truncated DEFLATE data.

1.8 DataFrame

A DataFrame contains unique, non-empty string `columns` and `rows` records.

1.8.1 Construction

1.8.1.1 `DF.from_rows(matrix [, opts])` `opts.header` uses the first row as names. `opts.columns` supplies explicit names when `header` is false.

```
local d = DF.from_rows({
  { "name", "age" },
  { "Alice", 30 },
  { "Bob", 25 },
}, { header = true })
```

Duplicate column names are rejected.

1.8.1.2 `DF.from_records(records [, columns])`

```lua
local d = DF.from_records({
  { name = "Alice", age = 30 },
  { name = "Bob", age = 25 },
}, { "name", "age" })
```

Without `columns`, string keys are inferred and sorted.

1.8.1.3 DF.from_sheet(sheet [, opts])

```lua
local d = DF.from_sheet(assert(wb:sheet("Data")), {
  header = true,
})
```

1.8.2 Introspection

```lua
print(d:nrow(), d:ncol())
print(table.concat(d:colnames(), ", "))
local ages = d:column("age")
```

`d:iter()` yields copies:

```lua
for row in d:iter() do
  row.age = 0 -- does not modify d
end
```

1.8.3 Non-destructive transformations

```lua
local adults = d:filter(function(row) return row.age >= 18 end)
local names = d:select("name")
local renamed = d:rename({ age = "years" })
local doubled = d:mutate("double_age", function(row) return row.age * 2 end)
local asc = d:sort("age")
local desc = d:sort("age", { desc = true })
local first = d:head(10)
local last = d:tail(10)
```

`filter` and `mutate` callbacks receive a copy. `filter`, `sort`, `head`, `tail` and `iter` do not share row tables with the source. Rename collisions and repeated selections are rejected. Sort is stable and keeps `nil` last.

1.8.4 Grouping and aggregation

```lua
local grouped = sales:groupby("city", "product"):agg({
  total = { "sum", "amount" },
  average = { "mean", "amount" },
  minimum = { "min", "amount" },
  maximum = { "max", "amount" },
  first = { "first", "amount" },
  last = { "last", "amount" },
  count = { "count" },
})
```

Functions: `sum`, `mean`/`avg`, `min`, `max`, `count`, `first`, `last`.

Group keys support nil, booleans, binary strings and numbers. The internal key encoding is type- and length-prefixed, so embedded separators or NUL bytes cannot merge distinct groups.

```lua
local counts = d:groupby("city"):count()
```

1.8.5 Output

```lua
local records = d:to_records()
local rows = d:to_rows()
local no_header = d:to_rows({ header = false })
```

```
print(d:tostring(20))
d:show(20)
```

1.9 Error contract

Caller mistakes raise Lua errors: invalid types or options, invalid names or dates, out-of-range indices and inconsistent DataFrame operations.

Filesystem and data failures normally return `nil, err`:

```
local wb, err = xlsx.open("missing.xlsx")
if not wb then
  io.stderr:write(err, "\n")
end
```

`workbook:sheet()` follows the same convention for a corrupt or missing sheet part. `Workbook:save()` checks writing, flushing, closing and publication and no longer reports success after a real write failure.

1.10 Tests and interoperability

```
./run_tests.sh
```

The harness covers write/read round trips, Unicode, XML entities, CRC corruption, 1900/1904 dates, Excel and XML limits, Babet calls, DataFrame copy semantics, collision-free grouping and real openpyxl interoperability.

Babet is resolved in this order: `BABET_BIN`, the local `bin/babet` file, then `babet` from `PATH`. The harness creates a temporary Python virtual environment, installs `openpyxl>=3.1,<4`, runs the interoperability round trip separately with Babet and standard Lua when both are available, and removes the venv, pip cache and temporary files at exit, including after a failure. A global openpyxl installation is neither used nor required.

This phase requires `python3`, the `venv` module, pip inside the venv and access to the configured Python package index. Debian and Ubuntu systems may need the `python3-venv` package.

```
cp /path/to/babet bin/babet
chmod +x bin/babet
./run_tests.sh

BABET_BIN=../babet/bin/babet ./run_tests.sh
LUA_BIN=/usr/bin/lua5.5 ./run_tests.sh
OPENPYXL_SPEC='openpyxl==3.1.5' ./run_tests.sh
```

1.11 Building the PDFs

```
cd doc
./build_doc.sh
```

Outputs:

- `documentation-fr.pdf`;
- `documentation-en.pdf`.

Pandoc and XeLaTeX or LuaLaTeX are required. Auxiliary LaTeX files are kept in a temporary directory.

1.12 Known limitations

- Writing uses STORED ZIP entries; reading supports STORED and DEFLATE.
- The internal Lua parser does not support ZIP64.
- No general visual styling, merged cells or formula writing.
- A formula's cached value can be read.
- No streaming reader: the file and selected XML parts remain in memory.
- The XML scanner is XLSX-specific, not a general XML parser.
- No Unicode normalization or complete Unicode case folding for sheet names; case-insensitive duplicate detection is reliable for ASCII.